Comparación de inserción directa y QuickSort en C++

Comparison of Insertion Sort and QuickSort in C++

René David Charaña Jalire

Universidad Nacional Jorge Basadre Grohmann, Tacna, Perú

Código: 2025-119017

Resumen. Se compararon los algoritmos de inserción directa y QuickSort mediante un programa en C++. Se probaron arreglos de 1 000, 5 000 y 10 000 números con datos aleatorios, ascendentes y descendentes. Cada caso se ejecutó 30 veces. Se registraron los tiempos con chrono y se guardaron los resultados en CSV. En total se obtuvieron 540 mediciones. QuickSort tuvo menores tiempos con números aleatorios y descendentes. Inserción directa fue más rápida cuando los números ya estaban ordenados. Se concluyó que la rapidez depende del algoritmo y del orden inicial de los datos.

Palabras clave: inserción directa, QuickSort, C++, ordenación, tiempo de ejecución.

Abstract. This study compared insertion sort and QuickSort using a C++ program. Arrays of 1,000, 5,000, and 10,000 numbers were tested with random, ascending, and descending data. Each case was run 30 times. Execution times were measured with chrono and saved in CSV files. A total of 540 measurements were collected. QuickSort was faster with random and descending data. Insertion sort was faster when the data were already in ascending order. The results showed that performance depends on the algorithm and on the initial order of the data.

Keywords: insertion sort, QuickSort, C++, sorting, execution time.

1. Introducción

Los algoritmos de ordenación permiten organizar números de menor a mayor. Hay varios métodos para hacerlo. Algunos son sencillos y otros trabajan mejor cuando aumenta la cantidad de datos. Inserción directa y QuickSort son dos ejemplos conocidos (Cormen et al., 2022).

El objetivo de este trabajo fue comparar ambos algoritmos en C++. Se buscó conocer cuánto tiempo demoran según la cantidad y el orden inicial de los números. También se registraron comparaciones y otros movimientos realizados por el programa.

2. Método

Se programaron los dos algoritmos en un solo archivo C++. Inserción directa coloca cada número en la posición que le corresponde. QuickSort usa como pivote el elemento del centro y ordena por partes. El código completo está en el repositorio de GitHub.

Se probaron tres tamaños: 1 000, 5 000 y 10 000 números. Se usaron tres patrones: aleatorio, ascendente y descendente. Cada combinación se repitió 30 veces. Ambos algoritmos recibieron copias de los mismos números para que la comparación fuera justa. Se utilizó la semilla 12345 para generar los datos aleatorios.

Los tiempos se midieron con chrono::steady_clock. El cronómetro se inició justo antes del ordenamiento y terminó después. La generación de números y la escritura del CSV quedaron fuera de esta medición. También se contaron comparaciones, movimientos e intercambios. En QuickSort se registró la profundidad máxima de recursión.

Las pruebas se realizaron en una computadora con Windows y un entorno de C++ Dev-C++. No se registraron en los datos disponibles el modelo del

procesador, la memoria RAM ni la versión exacta del compilador. El archivo experimental indica el commit 918a564 y la semilla 12345. Los resultados se guardaron en results/experimentos.csv.

3. Resultados

Se obtuvieron 540 registros: 270 de inserción directa y 270 de QuickSort. No se encontraron filas vacías ni repeticiones faltantes. La Tabla 1 muestra los tiempos promedio de las 30 repeticiones por cada condición.

Tabla 1. Tiempo promedio de los algoritmos (ms)

Cantidad	Patrón	Inserción	QuickSort
1 000	Aleatorio	0.9213	0.0946
1 000	Ascendente	0.0060	0.0255
1 000	Descendente	1.7327	0.0242
5 000	Aleatorio	24.2688	0.5833
5 000	Ascendente	0.0270	0.1292
5 000	Descendente	39.3802	0.1364
10 000	Aleatorio	82.2213	1.0388
10 000	Ascendente	0.0804	0.4064
10 000	Descendente	160.7190	0.2924

Nota. Un tiempo menor indica que el algoritmo terminó antes.

Con 10 000 datos aleatorios, inserción directa tardó 82.2213 ms en promedio y QuickSort 1.0388 ms. Con datos descendentes los tiempos fueron 160.7190 ms y 0.2924 ms. En cambio, con datos ascendentes inserción directa obtuvo 0.0804 ms y QuickSort 0.4064 ms.

La Tabla 2 presenta la media, la mediana, la desviación estándar (DE), el tiempo mínimo y el máximo para las pruebas con 10 000 números.

Tabla 2. Resumen de tiempos para 10 000 números (ms)

Algoritmo y patrón	Media	Mediana	DE	Mín.	Máx.
Inserción - Aleatorio	82.221	78.089	7.725	75.969	104.077
QuickSort - Aleatorio	1.039	0.999	0.125	0.965	1.520
Inserción - Ascendente	0.080	0.082	0.005	0.068	0.088
QuickSort - Ascendente	0.406	0.413	0.022	0.345	0.426
Inserción - Descendente	160.719	154.324	11.072	152.173	188.540
QuickSort - Descendente	0.292	0.284	0.028	0.279	0.426

Los valores de DE muestran que los tiempos no fueron idénticos en las 30 repeticiones. Por eso se utilizó el promedio para comparar los algoritmos.

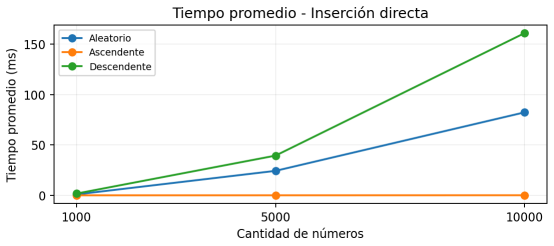


Figura 1. Tiempo promedio de inserción directa según la cantidad de datos.

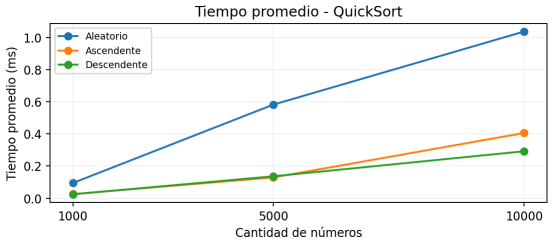


Figura 2. Tiempo promedio de QuickSort según la cantidad de datos.

4. Discusión

Los resultados muestran que inserción directa trabaja bien cuando los números ya están ordenados. En ese caso hace pocas

comparaciones y casi no necesita mover los elementos. Su tiempo puede crecer mucho cuando el arreglo está al revés. En el peor caso su trabajo crece aproximadamente como n^2 (Sedgewick y Wayne, 2011).

QuickSort fue más rápido con datos aleatorios y descendentes. En promedio suele trabajar cerca de n log n. Sin embargo, en algunos casos puede llegar a n^2. En este programa se eligió el pivote del centro (Cormen et al., 2022).

En los resultados de 10 000 números descendentes, inserción directa registró 49 995 000 comparaciones y QuickSort registró 125 452. Para ese mismo caso QuickSort tuvo 5 000 intercambios y una profundidad máxima de 13. Los contadores muestran el trabajo realizado por este código; no representan todas las operaciones de la computadora.

Inserción directa es un método estable: si hay dos valores iguales conserva su orden relativo. QuickSort, tal como se implementó aquí, no garantiza esa estabilidad. Este estudio no probó por separado un patrón con valores duplicados. Tampoco se compararon distintos procesadores o compiladores. Por eso los tiempos corresponden a las condiciones en que se ejecutó el programa.

5. Conclusiones

1. Los dos algoritmos ordenaron correctamente los datos de las pruebas realizadas.
2. QuickSort obtuvo menores tiempos con datos aleatorios y descendentes.
3. Inserción directa fue más rápida con datos que ya estaban ordenados en forma ascendente.
4. El tiempo de ordenamiento cambia según la cantidad de números y el orden inicial.